

排序算法

ALGORITHM	BUBBLE	SELECTION	INSERTION	HEAP	MERGE	QUICK
Time complexity	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$

Insertion Sort

```
void InsertionSort(vector<int>& arr, int len)
//arr为待排序数组，len为arr的长度，从小到大排序
{
    for(int i = 1; i < len; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while((j >= 0) && (key < arr[j]))
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

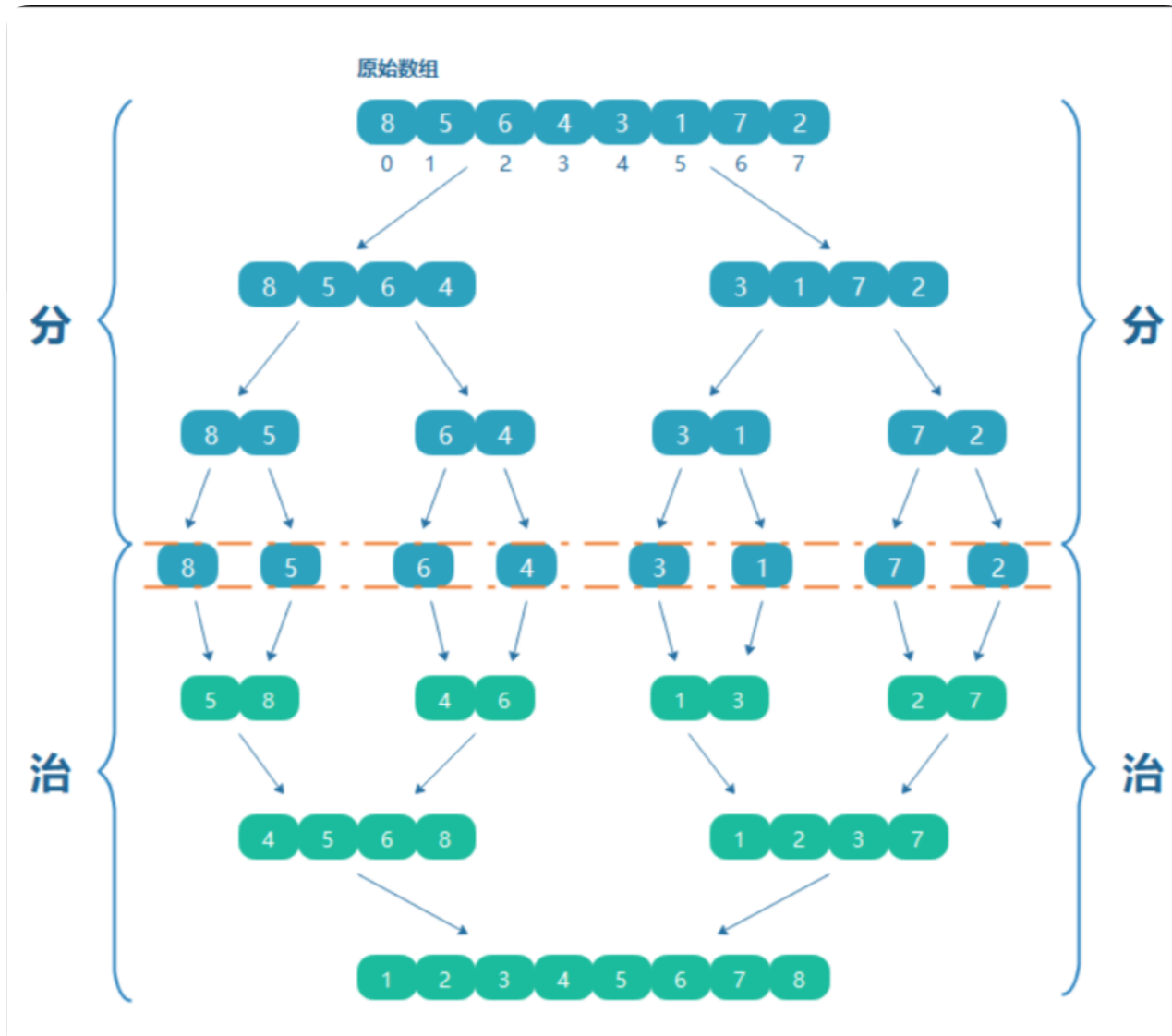
整体逻辑：

1. 选定一个key，最开始是数组的第二个元素（下标为1）
2. 让key和key前面的数组元素(即代码中的 `arr[j]`)进行大小比较，如果 `k < arr[j]`，即代表可以把key插在 `arr[j]` 的前面，此时进行 `arr[j + 1] = arr[j]`（将 `arr[j]` 前移一位）空出 `arr[j]` 的位置给key插入
3. 最终截止的时候 `arr[j+1] = key` 将空出的 `arr[j]` 的位置放置key（这里是 `arr[j + 1]` 因为在最后退出循环的时候进行了一次 `j--`）

可以把整体过程想象为给手里的扑克牌排序，从第二张扑克牌开始依次和前面的扑克牌比大小，直到当前选中的扑克牌是较大的那张，那么就把当前选中的扑克牌插在第一个比选中扑克牌小的牌后面(此时原来在选中扑克牌前面比它大的牌自动后移一位)



Merge Sort



Recursive version

```
void merge(vector<int>& arr, vector<int> temp, int left, int mid,
int right)
{
    int l_pos = left, r_pos = mid + 1, pos = left; //l_pos为左指针，开始指向左下标，r_pos为右指针，开始指向mid+1，pos作为temp数组的下标索引
    while(l_pos <= mid && r_pos <= right)
    {
        if(arr[l_pos] < arr[r_pos])
            temp[pos++] = arr[l_pos++];
        else
        {
            temp[pos++] = arr[r_pos++];
            result += mid - l_pos + 1; //这里的result计算的是逆序对的个数，初始化可为全局变量为0
        }
    }
    while(l_pos <= mid)
        temp[pos++] = arr[l_pos++];
    while(r_pos <= right)
        temp[pos++] = arr[r_pos++];
    while(left <= right)
    {
        arr[left] = temp[left];
        left++;
    } //或者采用函数copy(temp.begin() + left, temp.begin() + right + 1, arr.begin() + left);
}

void mergeSort(vector<int>& arr, vector<int>& temp, int left, int right) //left指数组开始下标，right为最后一个元素的下标
{
    if(left >= right) return;
    int mid = (right - left) / 2 + left; //避免溢出
    mergeSort(arr, left, mid); //处理左子串
    mergeSort(arr, mid+1, right); //处理右子串
    merge(arr, temp, left, mid, right); //合并
}
```

Iterative version

```
vector<int> merge(vector<int>& left, vector<int>& right)
{
    vector<int>temp;
    size_t l_len = left.size(), r_len = right.size();
    int l_pos = 0, r_pos = 0;
    while(l_pos < l_len && r_pos < r_len)
    {
        if(left[l_pos] < right[r_pos])
            temp.emplace_back(left[l_pos++]);
        else
            temp.emplace_back(right[r_pos++]);
    }
    while(l_pos < l_len)
        temp.emplace_back(left[l_pos++]);
    while(r_pos < r_len)
        temp.emplace_back(right[r_pos++]);
    return temp;
}

vector<int> mergeSort(const vector<int>& arr)
{
    deque<vector<int>> q;
    for(auto it : arr)
        q.emplace_back(vector<int>{it}); //这里相当于上图中的分
    while(q.size() > 1)
    {
        vector<int>left = q.front();
        q.pop_front();
        vector<int>right = q.front();
        q.pop_front();
        q.emplace_back(merge(left, right)); //相当于直接从图中的治的部分
    }
    return q.front();
}
```

Selection Sort

Basic idea: pick out minimum element from input, then recursively sort remaining elements, and finally concatenate the minimum element with sorted remaining elements.

SelectionSortRec(A):

```
if |A| = 1
    return A
else
    min := GetMinElement(A)
    A' := RemoveElement(A, min)
    return Concatenate(min, SelectionSortRec(A'))
```

SelectionSort(A):

```
for i := 1 to A.length
    minIdx := i
    for j := i + 1 to A.length
        if A[j] < A[minIdx]
            minIdx := j
    Swap(i, minIdx)
```

SelectionSortRecVariant(A):

```
if |A| = 1
    return A
else
    max := GetMaxElement(A)
    A' := RemoveElement(A, max)
    return Concatenate(SelectionSortRec(A'), max)
```

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Bubble Sort

Basic idea: repeatedly step through the array, compare adjacent pairs and swaps them if they are in the wrong order. Thus, larger elements "bubble" to the "top".

BubbleSort(A):

```
for i := A.length down to 2
    for j := 1 to i - 1
        if A[j] > A[j+1]
            Swap(A[j], A[j+1])
```



```

void BubbleSort(vector<int>& arr)
{
    int n = arr.size();
    for(int i = n-1; i > 0; i--)
    {
        for(int j = 0; j < i; j++)
        {
            if(arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

```

Time complexity: $\Theta(n^2)$

Space complexity: $\Theta(1)$

Improved Bubble Sort

```

BubbleSortImproved(A):
  n := A.length
  repeat
    swapped := false
    for j := 1 to n - 1
      if A[j] > A[j+1]
        Swap(A[j], A[j+1])
        swapped := true
    n := n - 1
  until swapped = false

```

Further Improved Bubble Sort

BubbleSortImprovedFurther(A):

$n := A.length$

repeat

$lastSwapIdx := -1$

 for $j := 1$ to $n - 1$

 if $A[j] > A[j+1]$

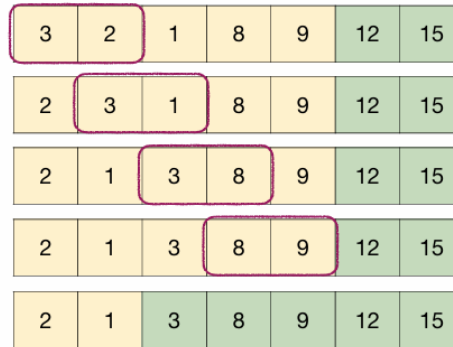
 Swap($A[j]$, $A[j+1]$)

$lastSwapIdx := j + 1$

$n := lastSwapIdx - 1$

until $n \leq 1$

$n = 5$



$lastSwapIdx = 2$

$lastSwapIdx = 3$

```
void BubbleSortImprovedFurther(vector<int>& arr)
{
    int n = arr.size();
    while(n > 1)
    {
        int lastSwapIndex = -1;
        for(int j = 0; j < n - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
                lastSwapIndex = j;
            }
        }
        n = lastSwapIndex + 1;
    }
}
```

*Shell Sort

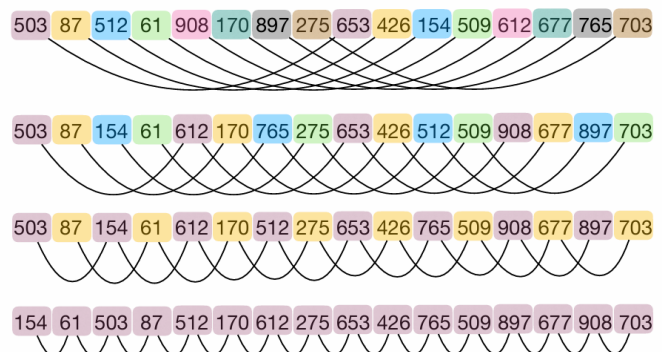
Let's first see an example of ShellSort: sort 16 integers.

[Pass 1] Group elements of distance 8 together, end up with eight groups each of size two. Sort these groups individually.

[Pass 2] Group elements of distance 4 together, end up with four groups each of size four. Sort these groups individually.

[Pass 3] Group elements of distance 2 together, end up with two groups each of size eight. Sort these groups individually.

[Pass 4] Group elements of distance 1, this is just an ordinary sort on all elements.



61 87 154 170 275 426 503 509 512 612 653 677 703 765 897 908

步骤：

- 1.先选定一个小于N的整数gap作为第一增量，然后将所有距离为gap的元素分在同一组，并对每一组的元素进行直接插入排序。然后再取一个比第一增量小的整数作为第二增量，重复上述操作...
- 2.当增量的大小减到1时，就相当于整个序列被分到一组，进行一次直接插入排序，排序完成。

```
//希尔排序
void ShellSort(int* arr, int n)
{
    int gap = n;
    while (gap>1)
    {
        //每次对gap折半操作
        gap = gap / 2;
        //单趟排序
        for (int i = 0; i < n - gap; ++i)
        {
            int end = i;
            int tem = arr[end + gap];
            while (end >= 0)
            {
                if (tem < arr[end])
                {
                    arr[end + gap] = arr[end];
                    end -= gap;
                }
                else
                {
                    break;
                }
            }
            arr[end + gap] = tem;
        }
    }
}
```

Time complexity: $O(n^2)$

Quick Sort

Basic idea:

Given an array A of n items.

1. Choose one item x in A as the pivot.
2. Use the pivot to partition the input into B and C , so that items in B are $\leq x$, and items in C are $> x$.
3. Recursively sort B and C .
4. Output $\langle B, x, C \rangle$

```
int InplacePartition(vector<int>& arr, int left, int right)
{
    int i = left - 1; // i指向的值左侧最后一个比arr[right]小的元素下标
    for(int j = left; j < right; j++)
    {
        if(arr[j] <= arr[right]) // 发现左侧有比arr[right]小的元素，就交换
arr[i+1]和arr[j]，因为arr[i+1]肯定比arr[j]要大
            swap(arr[++i], arr[j]);
    }
    swap(arr[i + 1], arr[right]);
    return i + 1; // 返回pivot的下标作为分组的标志
}

void QuickSort(vector<int>& arr, int left, int right) // left是arr的起
始下标，right是arr的最后一个元素的下标
{
    if(left < right)
    {
        int q = InplacePartition(arr, left, right);
        QuickSort(arr, left, q - 1);
        QuickSort(arr, q + 1, right);
    }
}
```

Time complexity: $O(n \lg n)$

C++中提供了相关的库函数

```
sort(iterator.begin, iterator.end, cmp);
```

cmp可传入，表示自定义的比较关系，返回值为bool，为true则是正确的排序关系，不传入的话，按照库函数中提供的比较函数进行比较

*External Sorting

Key (Simple) Idea

To find an element that is no larger than all elements in two lists, one only needs to compare minimum elements from each list

If:

$$A_1 \leq A_2 \leq \dots \leq A_n$$

$$B_1 \leq B_2 \leq \dots \leq B_m$$

Then:

$$\min(A_1, B_1) \leq A_i, \text{ for } 1 \leq i \leq n$$

$$\min(A_1, B_1) \leq B_j, \text{ for } 1 \leq j \leq m$$

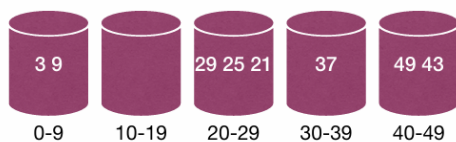
Each time put the current minimum elements back to disk

具体详情看课件6.排序 p72

Bucket Sort

- If the range of items' values is too large, allow each bucket to hold multiple values.
- Allocate k buckets each responsible for an interval of size d/k .
- But now we need to sort each bucket before combining them.

29 25 3 49 9 37 21 43



$n = 8, d = 50, k = 5$

BucketSort(A, k):

$\langle L_1, L_2, \dots, L_k \rangle = \text{CreateBuckets}(k)$

for $i := 1$ **to** $A.\text{length}$

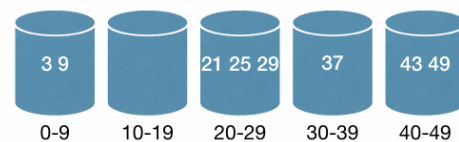
 AssignToBucket($A[i]$)

for $j := 1$ **to** k

 SortWithinBucket(L_j)

CombineBuckets($L_1, L_2, \dots, L_d$)

Sort in each bucket → 3, 9, 21, 25, 29, 37, 43, 49



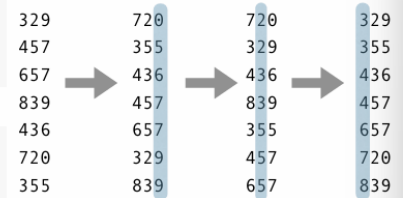
Radix Sort

RadixSort(A, d):

for $i := 1$ **to** d
 use-a-stable-sort-to-sort-A-on-digit- i

RadixSort(A, d):

for $i := 1$ **to** d
 use-**bucket-sort**-to-sort-A-on-digit- i



- Since only considering decimal numbers, we only need $10 = \Theta(1)$ buckets.
- RadixSort can sort n decimal d -digits numbers in $O(dn)$ time.

图的遍历（Graph Traversal）

引入：BFS(Breadth-First Search)、DFS(Depth-First Search)

图中的BFS遍历（默认边长为1）

```
void bfs(const vector<vector<int>>&edges, vector<int>&dist)//采用邻接表的形式记录node的neighbor_node
{
    queue<int>q;
    q.push(1);//默认1是sourceNode
    dist[1] = 0;
    while(!q.empty())
    {
        int current = q.front();
        q.pop();
        for(auto neighbor : edges[current])
        {
            if(dist[neighbor] == -1)
            {
                dist[neighbor] = dist[current] + 1;
                q.push(neighbor);
            }
        }
    }
}
```

```
}
```

图中的DFS遍历

```
void IterativeDfs(const vector<vector<int>>& edges, int start)
{
    vector<bool> visited(edges.size(), false);
    stack<int> stk;
    stk.push(start);
    while(!stk.empty())
    {
        int current = stk.top();
        stk.pop();
        if(!visited[current])
        {
            cout << current << ' ';
            visited[current] = true;
        }
        for(auto it = edges[current].rbegin(); it !=
edges[current].rend(); it++)
        {
            if(!visited[*it])
                stk.push(*it);
        }
    }
}
```

判断图中是否有环

```
bool dfs(const vector<vector<int>>& edges, vector<int>& visited,
int current)//正在访问为1,访问结束为2,visited初始化状态为0
{
    visited[current] = 1;
    for(auto neighbor : edges[current])
    {
        if(!visited[neighbor])
        {
            if(dfs(edges, visited, neighbor))
                return true;
        }
        else if(visited[neighbor] == 1)
            return true;
    }
    visited[current] = 2;
    return false;
}
```

```

        return true;
    }
    visited[current] = 2;
    return false;
}

```

What if the graph is not (strongly) connected?

- Do DFS from multiple sources.

```

DFSAll(G):
for each node  $u$  in  $V$ 
     $v.visited := False$ 
for each node  $u$  in  $V$ 
    if  $!u.visited$ 
        DFSSkeleton( $G, u$ )
DFSkeleton( $G, s$ ):
 $s.visited := True$ 
for each edge  $(s, v)$  in  $E$ 
    if  $!v.visited$ 
        DFSSkeleton( $G, v$ )

```

```

DFSAll(G):
for each node  $u$  in  $V$ 
     $v.visited := False$ 
for each node  $u$  in  $V$ 
    if  $!u.visited$ 
        DFSIterSkeleton( $G, u$ )
DFSIterSkeleton( $G, s$ ):
Stack  $Q$ 
 $Q.push(s)$ 
while  $!Q.empty()$ 
     $u := Q.pop()$ 
    if  $!u.visited$ 
         $u.visited := True$ 
        for each edge  $(u, v)$  in  $E$ 
             $Q.push(v)$ 

```

DAG（Directed Acyclic Graphs）有向无环图

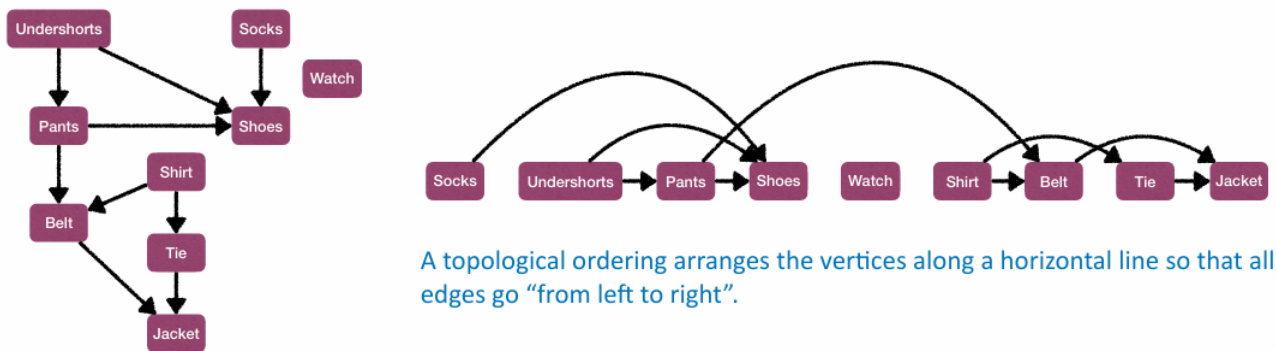
关于图中的算法：<https://oi-wiki.org/graph/shortest-path/#%E9%98%9F%E5%88%97%E4%BC%98%E5%8C%96spfa>

A graph without cycles is called acyclic.

A directed graph without cycles is a directed acyclic graph (DAG).

Topological Sort

A topological sort of a DAG G is a linear ordering of its vertices such that if G contains an edge (u, v) then u appears before v in the ordering.



Topological sort is impossible if the graph contains a cycle.

```
//基于dfs实现的topo排序
void dfs(const vector<vector<int>>& edges, stack<int>& stk, int
current, vector<bool>& visited)
{
    visited[current] = true;
    for(auto neighbor : edges[current])
    {
        if(!visited[neighbor])
            dfs(edges, stk, neighbor, visited);
    }
    stk.push(current);
}

vector<int> topologicalSort(const vector<vector<int>>& edges, int
numNodes)
{
    vector<bool>visited(numNodes, false);
    stack<int>stk;
    for(int i = 0;i < numNodes;i++)
    {
        if(!visited[i])
            dfs(edges, stk, i, visited);
    }
    vector<int>result;
    while(!stk.empty())
    {
        result.emplace_back(stk.top());
        stk.pop();
    }
    return result;
}
```

```

//基于bfs实现的topo排序
vector<int> topologicalSort(int numNodes, const
vector<vector<int>>& edges, vector<int>& penetration)
{
    queue<int>q;
    vector<int>result;
    for(int i = 1;i <= numNodes; i++)//根据具体情况选择从0开始还是从1开始
    {
        if(!penetration[i])//当入度减为0时可以进队
            q.push(i);
    }
    while(!q.empty())
    {
        int current = q.front();
        q.pop();
        result.emplace_back(current);
        for(auto neighbor : edges[current])
        {
            penetration[neighbor]--;
            if(!penetration[neighbor])
                q.push(neighbor);
        }
    }
    return result;
}

```

(Strongly) Connected Components

强连通分量

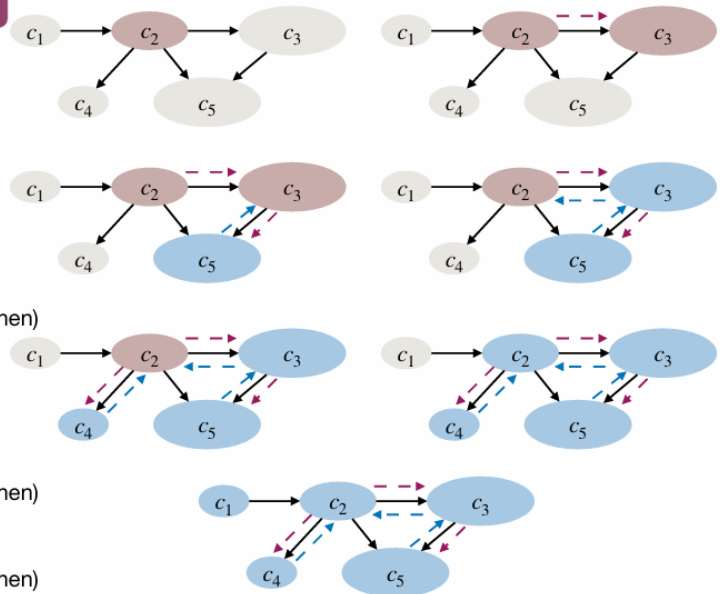
*Tarjan's SCC Algorithm

具体详情见课件14.深度优先应用p28

Tarjan's SCC Algorithm

Let's have a closer look at the order that DFS examines nodes

- First node in C_2 (root of C_2)
- Some nodes in C_2
- First node in C_3 (root of C_3)
- Some nodes in C_3
- First node in C_5 (root of C_5)
- All other nodes in C_5 (C_5 is a sink SCC)
- All other nodes in C_3 (C_3 becomes a sink SCC by then)
- Some nodes in C_2
- First node in C_4 (root of C_4)
- All other nodes in C_4 (C_4 is a sink SCC)
- All other nodes in C_2 (C_2 becomes a sink SCC by then)
- First node in C_1 (root of C_1)
- All other nodes in C_1 (C_1 becomes a sink SCC by then)



//使用dfs实现找出联通有向图的强连通分量

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

`int n, m, tot=0, cntb=0;` // `tot`表示发现此节点的时间, `cntb`表示强连通分量的分支数

`vector<vector<int>>edge, belong;` // `belong[i]`表示属于强连通分支*i*中的元素

`vector<int>dfn, low;` // `dfn`中存储发现此节点的时间 `low`作为每个点在这棵树中最小子树的根的访问时间

`vector<bool>inStack;` //表示该元素是否在栈中

`stack<int>s;`

```
void tarjan(int u)
```

```
{
```

```
    dfn[u] = low[u] = ++tot;
```

```
    s.push(u);
```

```
    inStack[u] = true;
```

```
    for(int v : edge[u])
```

```
    {
```

```
        if(!dfn[v])
```

```
        {
```

```
            // (1) 子节点更新 (DFS 树边)
```

```
            tarjan(v); // 递归访问未访问的邻居节点 v
```

```
            low[u] = min(low[u], low[v]); // 更新 u 的 low 值, 检查 v
```

的子树能否追溯到更早的祖先,此时已经回溯, `low[v]`已经更新好, 所以用`low[v]`

```
        }
```

```
        else if(inStack[v])
```



```

    {
        // (2) 当前路径祖先更新 (后退边)
        low[u] = min(low[u], dfn[v]); // 若 v 在栈中, u 通过后退边可以追溯到 dfn[v], 此时v的low可能还没更新, 所以要用dfn(v)
    }
}
if(low[u] == dfn[u])
{
    cntb++;
    int node;
    do{
        node = s.top();
        s.pop();
        instack[node] = false;
        belong[cntb].emplace_back(node);
    }while(node != u);
}
}

int main()
{
    cin >> n >> m;
    edge.resize(n+1), dfn.resize(n+1), low.resize(n+1),
    instack.resize(n+1, false), belong.resize(n);
    for(int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        edge[u].emplace_back(v);
    }
    for(int i = 1; i <= n; i++)
    {
        if(!dfn[i])
            tarjan(i);
    }
    cout << cntb << "\n";
    for(int i = 1; i <= cntb; i++)
    {
        cout << "强连通分量" << i << ": ";
        for(int node : belong[i])
            cout << node << ' ';
        cout << "\n";
    }
}

```

```
}  
}
```

Time complexity: $O(n + m)$, n 为顶点数, m 为边数

Minimum Spanning Trees (MST) 最小生成树

A spanning tree is a tree containing all nodes in V and a subset T of all the edges E .

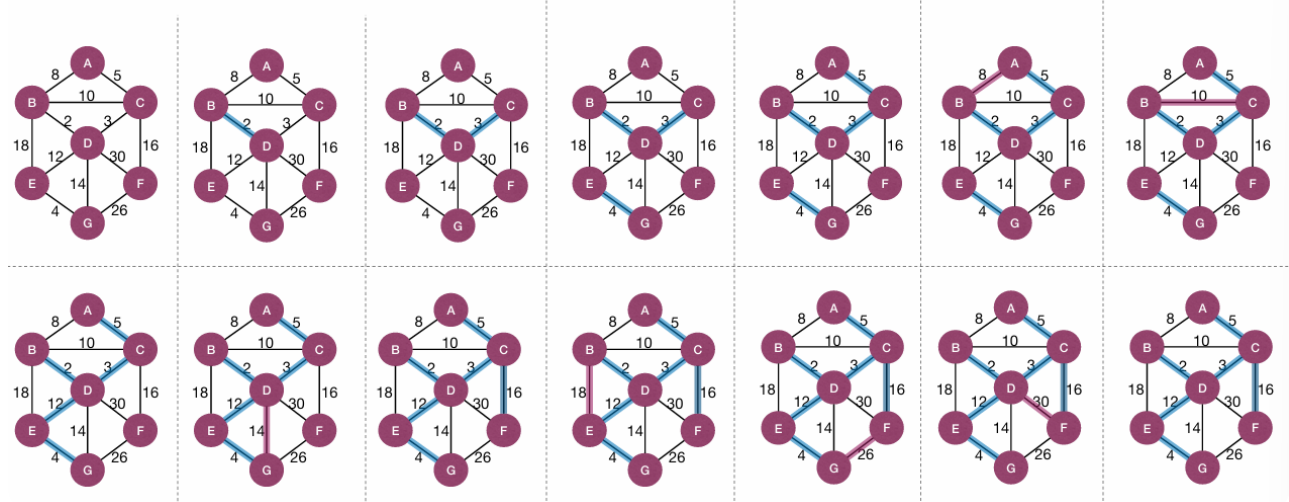
A minimum spanning tree (MST) is a spanning tree whose total weight $w(T) = \sum_{(u,v) \in T} w(u,v)$ is minimized.

Kruskal's Algorithm

Time Complexity: $O(m \log n)$ when using disjoint set data structure

适合用于稀疏图

- Edgen weights in increasing order: 2 3 4 5 8 10 12 14 16 18 26 30



KruskalMST(G,w):

$A := \emptyset$

Sort edges into weight increasing order

for each edge (u,v) taken in weight increasing order

if adding edge (u,v) does not form cycle in A

$A := A \cup \{(u, v)\}$

return A

KruskalMST(G,w):

$A := \emptyset$

Sort edges into weight increasing order

$O(m \log m) = O(m \log n)$

for each node u in V

MakeSet(u)

$O(n)$

for each edge (u,v) taken in weight increasing order

if $\text{Find}(u) \neq \text{Find}(v)$

$A := A \cup \{(u, v)\}$

Union(u, v)

$O(m \log^* n)$

return A

$$m \leq n^2$$

- Runtime of Kruskal's algorithm?
 - $O(m \log n)$ when using disjoint-set data structure

Codes:

```
#include<bits/stdc++.h>
using namespace std;

struct edge{
    int u, v, w;
    edge(int _u, int _v, int _w) : u(_u), v(_v), w(_w) {}
};

int n, m; //顶点总数
vector<edge>edges;
vector<int>pre, weight; //使用前记得进行初始化
```

```

bool cmp(const edge& a, const edge& b)
{
    return a.w < b.w;
}

int findPre(int x)
{
    if(pre[x] == x)
        return x;
    return pre[x] = findPre(pre[x]);
}

bool Union(int x, int y)
{
    int pre_x = findPre(x), pre_y = findPre(y);
    if(pre_x == pre_y)
        return false;
    if(weight[pre_x] < weight[pre_y])
        pre[pre_x] = pre_y;
    else
    {
        if(weight[pre_x] == weight[pre_y])
            weight[pre_x]++;
        pre[pre_y] = pre_x;
    }
    return true;
}

int kruskal()
{
    int result = 0, edgeCount = 0;
    sort(edges.begin(), edges.end(), cmp);
    for(auto edge : edges)
    {
        auto [u, v, w] = edge;
        if(Union(u, v))
            result += w, edgeCount++;
        if(edgeCount == n - 1)
            break;
    }
    if(edgeCount == n - 1)
        return result;
}

```

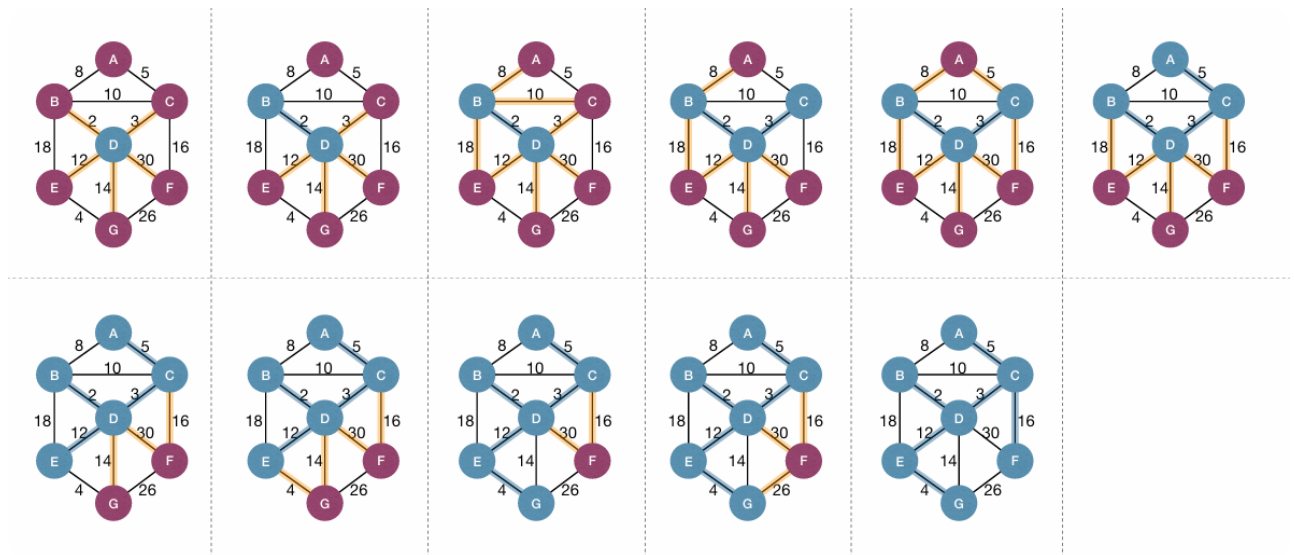
```

else
    return -1;
}
int main()
{
    cin >> n >> m;
    pre.resize(n+1), weight.resize(n + 1, 0);
    for(int i = 1; i <= n; i++)
        pre[i] = i;
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
        edges.emplace_back(v, u, w);
    }
    cout << kruskal();
}

```

Prim's Algorithm

适合用于稠密图



PrimMST(G,w):

$O(m \lg n)$ using binary heap to implement priority queue

$x := \text{Pick an arbitrary node in } G$

for each node u in V

$u.dist := INF, u.parent := NIL, u.in := False$

$O(n)$

$x.dist := 0$

PriorityQueue $Q := \text{Build a priority queue based on "dist" values}$

$O(n)$

while Q is not empty

$u := Q.ExtractMin()$

$u.in := True$

$O(n \lg n)$

for each edge (u,v) in E

if $v.in = False$ and $w(u,v) < v.dist$

$v.parent := u, v.dist := w(u,v)$

$Q.Update(v, w(u,v))$

$O(m \lg n)$

Could be faster using better priority queue implementation (By using fibonacci heaps instead)

```
#include<bits/stdc++.h>
using namespace std;

int n, m; // n为定点数, m为边数
vector<vector<pair<int, int>>>edges; // pair.first = v, pair.second = weight;
struct Compare{
    bool operator()(const pair<int, int>& a, const pair<int, int>& b) const {
        return a.second > b.second;
    }
};
vector<bool>inTree;
int prime()
{
    int result = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, Compare>minHeap; // pair.first = v, pair.second = w;
    minHeap.emplace(1, 0);
    while(!minHeap.empty())
    {
        auto [u, w] = minHeap.top();
        minHeap.pop();
        if(inTree[u])
            continue;
        inTree[u] = true;
```

```

        result += w;
        for(auto [neighbor, neighbor_weight] : edges[u])
        {
            if(!inTree[neighbor])
                minHeap.emplace(neighbor, neighbor_weight);
        }
    }
    for(int i = 1; i <= n; i++)
    {
        if(!inTree[i])
            return -1;
    }
    return result;
}

int main()
{
    cin >> n >> m;
    edges.resize(n + 1), inTree.resize(n + 1, false);
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges[u].emplace_back(v, w);
        edges[v].emplace_back(u, w);
    }
    cout << prime();
}

```

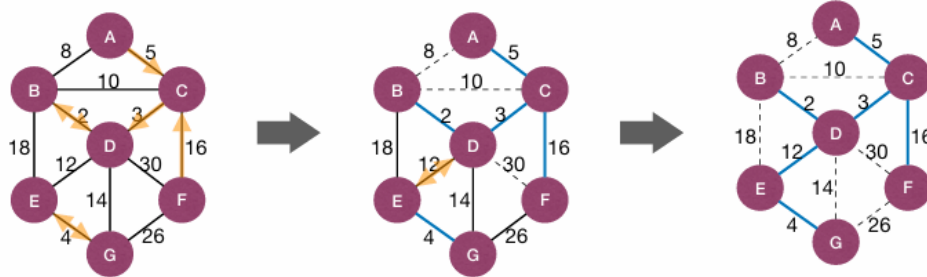
Time complexity : $O(m \cdot \lg n)$, 如果采用数组来记录minWeight则复杂度为 $O(n^2)$

Borůvka's Algorithm

Starting with all nodes and an empty set of edges A .

Find **MWOE** for every remaining CC in G_A , add all of them to A .

Repeat above step until we have a spanning tree.



KruskalMST(G, w):

→ Total runtime is $O(m \lg n)$

$G' := (V, \emptyset)$

do

$ccCount := \text{CountCCAndLabel}(G')$

→ $O(n)$ // Do DFS/BFS, count # of CC, give **ccNum** to nodes.

for $i := 1$ to $ccCount$

$safeEdge[i] := NIL$

→ $O(n)$

for each edge (u, v) in $E(G)$

if $u.ccNum \neq v.ccNum$

if $safeEdge[u.ccNum] = NIL$ or $w(u, v) < w(safeEdge[u.ccNum])$

$safeEdge[u.ccNum] := (u, v)$

if $safeEdge[v.ccNum] = NIL$ or $w(u, v) < w(safeEdge[v.ccNum])$

$safeEdge[v.ccNum] := (u, v)$

→ $O(m + n) = O(m)$

for $i := 1$ to $ccCount$

Add $safeEdge[i]$ to $E(G')$

→ $O(n)$

while $ccCount > 1$

→ $O(\lg n)$ interactions

WHY?

return $E(G')$

```
#include<bits/stdc++.h>
using namespace std;

int n, m;

struct edge{
    int u, v, w;
    edge(int from, int to, int weight) : u(from), v(to), w(weight)
}

vector<edge>edges;
vector<int>pre, weight;
int findPre(int x)
{
    if(pre[x] == x)
```



```

        return pre[x];
    return pre[x] = findPre(pre[x]);
}

void Union(int u, int v)
{
    int pre_u = findPre(u), pre_v = findPre(v);
    if(pre_u == pre_v)
        return;
    if(weight[pre_u] < weight[pre_v])
        pre[pre_u] = pre_v;
    else
    {
        if(weight[pre_u] == weight[pre_v])
            weight[pre_u]++;
        pre[pre_v] = pre_u;
    }
}

int boruvka()
{
    vector<int>cheapest(n+1, -1); //每个分量的最小边
    int result = 0, numComponents = n;
    while(numComponents > 1)
    {
        //找到每个分量的最小外向边
        for(int i = 0; i < edges.size(); i++)
        {
            auto [u, v, w] = edges[i];
            int root_u = findPre(u), root_v = findPre(v);
            if(root_u != root_v)
            {
                if(cheapest[root_u] == -1 || w <
edges[cheapest[root_u]].w)
                    cheapest[root_u] = i;
                if(cheapest[root_v] == -1 || w <
edges[cheapest[root_v]].w)
                    cheapest[root_v] = i;
            }
        }
        //将最小边加入最小生成树
        for(int i = 1; i <= n; i++)
        {
            if(cheapest[i] != -1)

```

```

        {
            auto[u, v, w] = edges[cheapest[i]];
            int root_u = findPre(u), root_v = findPre(v);
            if(root_u != root_v)
            {
                Union(root_u, root_v);
                result += w;
                numComponents--;
            }
        }
    }
}
return result;
}
int main()
{
    cin >> n >> m;
    pre.resize(n + 1), weight.resize(n + 1, 0);
    for(int i = 1; i <= n; i++)
        pre[i] = i;
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
        edges.emplace_back(v, u, w);
    }
    cout << boruvka();
}

```

Time complexity : $O(m \log n)$

总结：

Kruskal's algorithm 和 Prim's algorithm 其实都采用了贪心策略

Kruskal's algorithm 更适用于稀疏图，因为排序算法的时间复杂度是 $O(m \log m)$ ， m 远小于 n^2 ，且 $m \geq n - 1$ ，所以时间复杂度为 $O(m \log m) + O(n) + O(m \log n) = O(m \log m)$ ，与边数相关

Prim's algorithm 更适用于稠密图，此时 m 接近于 n^2 ，此时与顶点数的关系比较大

Borůvka's Algorithm在稀疏图（边数远小于顶点数平方）或需要分布式计算的场景中表现优异

Single-Source Shortest Path (SSSP) 单源最短路径

The SSSP Problem: Given a graph $G = (V, E)$ and a weight function w , given a source node s , find a shortest path from s to every node $u \in V$.

The graph can be directed.

Negative edge weight allowed.

Negative cycle not allowed.

Dijkstra's algorithm

attention: Dijkstra's algorithm doesn't work when there is negative edge weight in the graph.

Time complexity : $O((n + m) \cdot \log n)$ when using a binary heap.

$O(m \log m)$ when using priority queue;

DijkstraSSSP(G, s):

for each u **in** V

$u.dist := INF, u.parent := NIL$

$O(n)$

$s.dist := 0$

Build priority queue Q based on $dist$

$O(n)$

while $!Q.empty()$

$u := Q.ExtractMin()$

$O(n \log n)$

for each edge (u, v) **in** E

if $v.dist > u.dist + w(u, v)$

$v.dist := u.dist + w(u, v)$

$O(m \log n)$

$v.parent := u$

$Q.UpdateKey(v)$

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n, m; //n为顶点数, m为边数
```

```
vector<vector<pair<int, int>>>edges; //pair.first = v, pair.second = w;
```

```
vector<int>parent, dist;
```

```
vector<bool>visited;
```

```
void Dijkstra(int sr)
```

```
{
```

```
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>>minHeap;
```

```
    dist[sr] = 0;
```

```
    minHeap.emplace(dist[sr], sr);
```

```
    while(!minHeap.empty())
```

```
    {
```

```
        int u = minHeap.top().second;
```

```
        minHeap.pop();
```

```

        if(visited[u])
            continue;
        visited[u] = true; //Dijkstra是不会遍历已经确认dist的node的，如果
        加visited的判断的话，将会是SPFA (Shortest Path Faster Algorithm)
        for(auto edge : edges[u])
        {
            auto [v, w] = edge;
            if(dist[v] > dist[u] + w)
            {
                dist[v] = dist[u] + w;
                parent[v] = u;
                minHeap.emplace(dist[v], v);
            }
        }
    }
}

void printPath(int sr, int des)
{
    if(parent[des] == -1)
        return;
    vector<int>path;
    while(des != sr)
    {
        path.emplace_back(des);
        des = parent[des];
    }
    path.emplace_back(sr);
    reverse(path.begin(), path.end());
    for(auto it : path)
        cout << it << ' ';
    cout << endl;
}

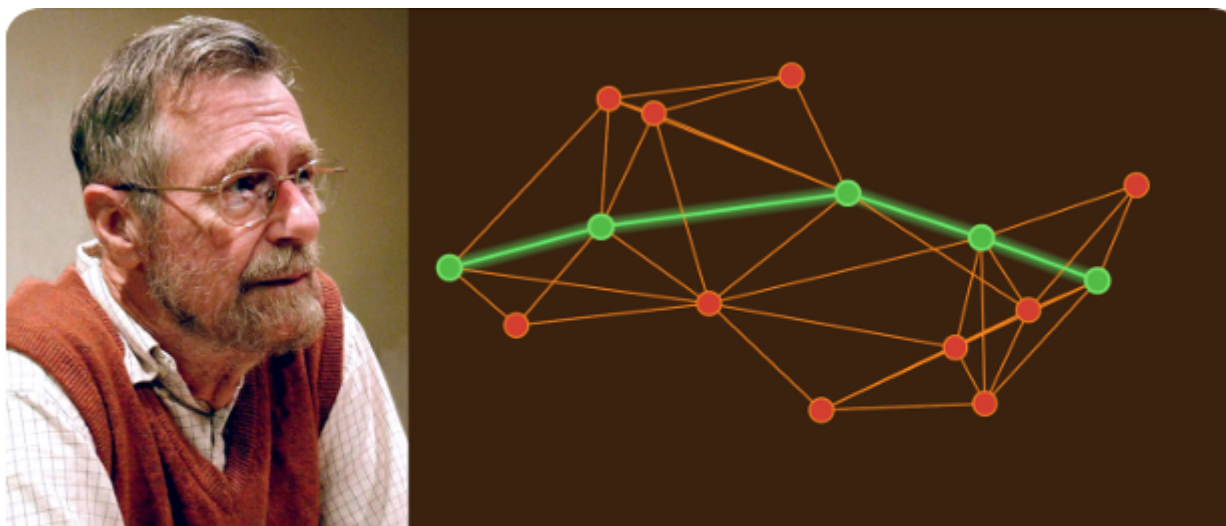
int main()
{
    cin >> n >> m;
    edges.resize(n+1), parent.resize(n+1, -1),
        dist.resize(n+1, INT_MAX/2), visited.resize(n + 1,
false);
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
    }
}

```

```
        edges[u].emplace_back(v, w);
        edges[v].emplace_back(u, w);
    }
    Dijkstra(1);
    printPath(1, n);
}
```

续：题中说明了没有负边的时候尽量用Dijkstra，因为SPFA算法能够退化到 $O(n \cdot m)$ 的复杂度，如果有负边那就采用Bellman-Ford算法

关于SPFA算法https://blog.csdn.net/qq_14886903/article/details/122891795?fromshare=blogdetail&sharetype=blogdetail&sharerId=122891795&sharerefer=PC&sharesource=2303_79715771&sharefrom=from_link



本科经典算法Dijkstra，被证明是普遍最优了：最坏情况性能也最优！



量子位

2020 年度新知答主

已关注

1100 人赞同了该文章

金磊 发自 凹非寺

量子位 | 公众号 QbitAI

时隔近70年，那个用来解决最短路径问题的经典算法——Dijkstra，现在有了新突破：

被证明具有普遍最优性 (Universal Optimality)。

什么意思？

这就意味着不论它面对多复杂的图结构，即便在最坏情况下都能达到理论上的最优性能！

而且这还是学术界首次将这一概念应用于任何序列算法。

Bellman-Ford Algorithm

适合于含有负边的图，并且可以检查出是否含有负环

Time complexity : $\Theta(n(m + n))$, $O(n \cdot m)$ 也不是不行

BellmanFordSSSP(G, s):

for each u in V

$u.dist := INF, u.parent := NIL$

$s.dist := 0$

repeat $n - 1$ times

for each edge (u, v) in E

if $v.dist > u.dist + w(u, v)$

$v.dist := u.dist + w(u, v)$

$v.parent := u$

遍历所有边 $n-1$ 次肯定可以找到最短路径



- Consider a shortest path from s to v .
 - ▶ **Observation 1:** if **Update** (s, u_1), **Update** (u_1, u_2), ..., **Update** (u_{k-1}, u_k), **Update** (u_k, v) are executed, then we correctly obtain the shortest path.
 - ▶ **Observation 2:** in above sequence, before and after each **Update**, we can add arbitrary **Update** sequence, and still get shortest path from s to v .
- Algorithm: simply **Update** all edges, for $k + 1$ times!





- But how large is $k + 1$?
 - **Observation 3:** any shortest path cannot contain a cycle. (WHY?)
- Algorithm: simply Update all edges, for $n - 1$ times!
 - The Bellman-Ford Algorithm!

```
#include<bits/stdc++.h>
using namespace std;

int n, m; //n为顶点数, m为边数
struct edge{
    int u, v, w;
    edge (int from, int to, int weight) : u(from), v(to), w(weight)
    {}
};
vector<edge>edges;
vector<int>parent, dist, backup;

bool BellmanFord(int sr) //含有负环则返回true
{
    dist[sr] = 0;
    for(int i = 0; i < n - 1; i++)
    {
        backup = dist;
        for(auto edge : edges)
        {
            auto [u, v, w] = edge;
            if(dist[v] > dist[u] + w)
            {
                dist[v] = dist[u] + w;
                parent[v] = u;
            }
        }
    }
    for(auto [u, v, w] : edges)
    {
        if(dist[v] > dist[u] + w)
```

```

        return true;
    }
    return false;
}
void printPath(int sr, int des)
{
    if(parent[des] == -1)
        return;
    vector<int>path;
    while(des != sr)
    {
        path.emplace_back(des);
        des = parent[des];
    }
    path.emplace_back(sr);
    reverse(path.begin(), path.end());
    for(auto it : path)
        cout << it << ' ';
    cout << endl;
}
int main()
{
    cin >> n >> m;
    parent.resize(n+1, -1), dist.resize(n+1, INT_MAX/2);
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
    }
    BellmanFord(1);
    printPath(1, n);
}

```

SSSP in DAG (with negative weights)

$O(m + n)$ time complexity

DAGSSSP(G, s):

for each u **in** V

$u.dist := INF, u.parent := NIL$

$s.dist := 0$

Run DFS to obtain topological order

for each *node* u **in** *topological order*

for each *edge* (u, v) **in** E

if $v.dist > u.dist + w(u, v)$

$v.dist := u.dist + w(u, v)$

$v.parent := u$

```
#include<bits/stdc++.h>
using namespace std;

int n, m; //n为顶点数, m为边数
vector<vector<pair<int, int>>>edges;
vector<int>dist, parent;

void Dfs(int current, vector<bool>&visited, stack<int> &topoStack)
{
    visited[current] = true;
    for(auto [v, w] : edges[current])
        if(!visited[v])
            Dfs(v, visited, topoStack);
    topoStack.push(current);
}

void DAGSSSP(int s)
{
    dist[s] = 0;
```

```

vector<bool>visited(n + 1, false);
stack<int>topoStack;
for(int i = 1; i <= n; i++)
    if(!visited[i])
        Dfs(i, visited, topoStack);
while(!topoStack.empty())
{
    int u = topoStack.top();
    topoStack.pop();
    if(dist[u] != INT_MAX)
    {
        for(auto [v, w] : edges[u])
            if(dist[v] > dist[u] + w)
                dist[v] = dist[u] + w, parent[v] = u;
    }
}

void printPath(int sr, int des)
{
    if(parent[des] == -1)
        return;
    vector<int>path;
    while(des != sr)
    {
        path.emplace_back(des);
        des = parent[des];
    }
    path.emplace_back(sr);
    reverse(path.begin(), path.end());
    for(auto it : path)
        cout << it << ' ';
    cout << endl;
}

int main()
{
    cin >> n >> m;
    parent.resize(n+1, -1), dist.resize(n+1, INT_MAX),
    edges.resize(n+1);
    for(int i = 0; i < m; i++)
    {

```

```

    int u, v, w;
    cin >> u >> v >> w;
    edges[u].emplace_back(v, w);
}
DAGSSSP(1);
printPath(1, n);
}

```

All-Pairs Shortest Path(APSP)全源最短路径

All-Pairs Shortest Paths (APSP) Problem:

- Given a graph $G = (V, E)$ and a weight function w , for **every pair** $(u, v) \in V \times V$, find a shortest path from u to v .

Johnson's Algorithm

可以选择跑 n 次Bellman-Ford, 但此时复杂度上升为 $O(n^2 \cdot m)$, 比较好的是通过添加一个虚节点, 然后跑 n 次Dijkstra, 此时复杂度为 $O(n \cdot m \log m)$

Re-weight edges:

$\hat{w}(u, v) = \text{dist}(u) + w(u, v) - \text{dist}(v) \geq 0$ Create $H := (V + \{z\}, E + \{(z, v) \mid v \in V\})$ with $w(z, v) = 0$
Bellman-FordSSSP(H, z) to obtain dist_H

- For node pairs in G , addition of z does not create new shortest path.
- For node pairs in G , a path is shortest under w iff this path is shortest under $\hat{w}$.

JohnsonAPSP(G, s):

for each edge (u, v) **in** $H.E$

$w'(u, v) := \text{dist}_H(z, u) + w(u, v) - \text{dist}_H(z, v)$

for each node u **in** $G.V$

DijkstraSSSP(G, u) with w' to obtain $\text{dist}_{G, w'}$

for each node v **in** $G.V$

$\text{dist}_G(u, v) := \text{dist}_{G, w'}(u, v) + \text{dist}_H(z, v) - \text{dist}_H(z, u)$

Proposed by Donald Bruce Johnson

```

#include <bits/stdc++.h>
using namespace std;
const int INF = INT_MAX;

// 边定义
struct Edge {
    int u, v, w;

```

```

    Edge(int from, int to, int weight) : u(from), v(to), w(weight)
    {}
};

// 全局变量
int n, m; // n 为节点数, m 为边数
vector<Edge> edges; // 边列表
vector<vector<pair<int, int>>> adjList; // 邻接表

// Bellman-Ford 算法, 判断负权环并计算 h 值
bool bellmanFord(int s, vector<int> &h) {
    h.assign(n + 1, INF);
    h[s] = 0;
    for (int i = 1; i <= n; i++) {
        for (auto &edge : edges) {
            int u = edge.u, v = edge.v, w = edge.w;
            if (h[u] != INF && h[v] > h[u] + w) {
                h[v] = h[u] + w;
                if (i == n) return false; // 负权环存在
            }
        }
    }
    return true;
}

// Dijkstra 算法
vector<int> dijkstra(int s) {
    vector<int> dist(n + 1, INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<>> minHeap;

    dist[s] = 0;
    minHeap.emplace(0, s);

    while (!minHeap.empty()) {
        auto [d, u] = minHeap.top();
        minHeap.pop();
        if (d > dist[u]) continue;

        for (auto &[v, w] : adjList[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
            }
        }
    }
    return dist;
}

```

```

        minHeap.emplace(dist[v], v);
    }
}
return dist;
}

// Johnson 算法
bool johnson() {
    // 添加虚拟节点 s，连接到每个节点，权值为 0
    for (int i = 1; i <= n; i++) {
        edges.emplace_back(0, i, 0);
    }

    // 运行 Bellman-Ford 算法
    vector<int> h;
    if (!bellmanFord(0, h)) {
        cout << "Graph contains a negative weight cycle." << endl;
        return false;
    }

    // 重新调整边权
    for (auto &edge : edges) {
        edge.w += h[edge.u] - h[edge.v];
    }

    // 使用调整后的权重构造邻接表
    adjList.assign(n + 1, {});
    for (auto &edge : edges) {
        adjList[edge.u].emplace_back(edge.v, edge.w);
    }

    // 运行 Dijkstra 算法
    for (int i = 1; i <= n; i++) {
        vector<int> dist = dijkstra(i);
        cout << "Shortest paths from node " << i << ":\n";
        for (int j = 1; j <= n; j++) {
            if (dist[j] == INF)
                cout << "INF ";
            else
                cout << dist[j] + h[j] - h[i] << " "; // 恢复原权重
        }
    }
}

```

```

        cout << endl;
    }
    return true;
}

int main() {
    cin >> n >> m;
    edges.reserve(m);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
    }

    if (!johnson()) {
        cout << "Failed to compute shortest paths." << endl;
    }
    return 0;
}

```

详细证明见:<https://oi-wiki.org/graph/shortest-path/#johnson-%E5%85%A8%E6%BA%90%E6%9C%80%E7%9F%AD%E8%B7%AF%E5%BE%84%E7%AE%97%E6%B3%95>

Floyd-Warshall

Time complexity: $O(n^3)$

$$\bullet \quad dist(u, v, r) = \begin{cases} w(u, v) & \text{if } r = 0 \text{ and } (u, v) \in E \\ \infty & \text{if } r = 0 \text{ and } (u, v) \notin E \\ \min \begin{cases} dist(u, v, r-1) \\ dist(u, x_r, r-1) + dist(x_r, v, r-1) \end{cases} & \text{otherwise} \end{cases}$$



Bernard Roy



Robert W. Floyd



Stephen Warshall

FloydWarshallAPSP(G):

```

for each pair (u,v) in V*V
    if (u, v) in E then dist[u,v, 0] := w(u, v)
    else dist[u,v,0] := INF
for r := 1 to n
    for each node u
        for each node v
            dist[u,v,r] := dist[u,v,r - 1]
            if dist[u,v,r] > dist[u,x_r, r - 1] + dist[x_r,v, r - 1]
                dist[u,v,r] := dist[u,x_r, r - 1] + dist[x_r,v, r - 1]

```

Time complexity:
 $O(n^3)$


```

#include<bits/stdc++.h>
using namespace std;

int n, m; //n为顶点数, m为边数
vector<vector<int>>>dist;
void Floydwarshall()
{
    for(int k = 1; k <= n; k++)
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);
}

int main()
{
    cin >> n >> m;
    dist.resize(n + 1, vector<int>(n + 1, INT_MAX/2));
    for(int i = 1; i <= n; i++)
        dist[i][i] = 0;
    for(int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w);
        dist[v][u] = min(dist[v][u], w);
    }
    Floydwarshall();
}

```

证明: https://blog.csdn.net/ljhandlwt/article/details/52096932?fromshare=blogdetail&sharetype=blogdetail&sharerId=52096932&sharerefer=PC&sharesource=2303_79715771&sharefrom=from_link